

TRABAJO PRÁCTICO

Programación Orientada a Objetos

TP ETAPA 1

Profesor: Leon María Angela

Integrantes:
Aranguren Alejo - 1215686
Raggio Gonzalo - 1214922
Schipani Tiziano - 1214840

Turno tarde

2.º cuatrimestre de 2026

Análisis del enunciado

En este proyecto nos enfocaremos en el desarrollo de la propuesta C. La misma destaca un sistema de movilidad compartida el cual incluye bicicletas y monopatines distribuidos en la ciudad con el objetivo de facilitar el movimiento de los ciudadanos.

Enlace al repositorio de GitHub: tizisch/TP_POO_OBLIGATORIO

Actores:

Los actores identificados para este proyecto son usuario, operador del sistema, técnico de mantenimiento y administrador.

Reglas de negocio:

Usuarios y saldo

• Un usuario solo puede retirar un vehículo si su saldo alcanza el mínimo establecido (por ejemplo, el equivalente a 30 minutos de uso) y si no tiene otro viaje abierto.

• Si al terminar un viaje el saldo queda negativo, el usuario queda inhabilitado hasta regularizarlo.

Vehículos

• Hay tres tipos: bicicleta común, bicicleta eléctrica y monopatín eléctrico. Los estados son disponible, en uso, en reparación, batería baja y fuera de servicio.

• Solo un vehículo disponible puede retirarse. Los eléctricos con batería menor al 15% pasan a batería baja y no se ofrecen. Las bicicletas comunes no tienen batería.

• Un vehículo con 3 reportes de falla en 30 días pasa a mantenimiento obligatorio.

• Una orden de mantenimiento no se cierra sin registrar el técnico, la fecha y las tareas realizadas.

Estaciones

• Cada estación tiene una capacidad fija, y no se puede devolver un vehículo en una estación llena ni fuera de una estación.

• Si el usuario encuentra la estación llena, dispone de 10 minutos adicionales sin cargo para llegar a otra cercana.

• Una estación con menos del 20 % o más del 90 % de ocupación genera una alerta de redistribución para el operador.

Viajes y tarifas

• La tarifa es un cargo de desbloqueo más un precio por minuto según el tipo de vehículo. Valores de referencia: bicicleta común $200 + $20 por minuto, bicicleta eléctrica $300 + $45 por minuto, monopatín $300 + $50 por minuto.

• Un retiro cancelado dentro de los 2 minutos no tiene cargo.

• El tiempo máximo de un viaje es de 2 horas. Pasado ese límite, el minuto se cobra con un recargo del 50 %. Si el vehículo no se devuelve en 24 horas, se lo reporta como “fuera de servicio” y se aplica un cargo fijo.

• Promociones: el primer viaje de cada usuario es gratis hasta 15 minutos, y el plan mensual incluye 20 viajes de hasta 30 minutos. No se acumulan: se aplica la más favorable para el usuario.

• Un viaje cerrado no se modifica. Cualquier corrección la hace el operador con una nota de ajuste.

Requerimientos funcionales:

Requerimientos relacionados con el Usuario

1. Registrar usuario
2. Registrar medio de pago
3. Agregar saldo a cuenta
4. Suscribirse a plan mensual
5. Reportar falla
6. Retirar vehículo
7. Devolver vehículo
8. Calcular tarifa
9. Aplicar descuento

Requerimientos de Operador

1. Registrar fallas
2. Generar orden de mantenimiento(extends de registrar fallas)
3. Redistribuir vehículos entre estaciones

Requerimientos de Administrador

1. Emitir reporte de viaje por usuario
2. Emitir reporte de uso por estación
3. Emitir reporte de vehículos con más fallas

Requerimientos de Técnico de mantenimiento

1. Cerrar orden de mantenimiento (Legajo del técnico, registro de fecha, tareas realizadas y repuestos utilizados).

Supuestos adoptados

1. Establecemos un saldo mínimo dinámico según el tipo de vehículo, para esto el sistema validará que el usuario cuente con el costo mínimo para retirar un vehículo y usarlo por lo menos 30 minutos, por ejemplo el usuario quiere retirar una bicicleta por lo que el sistema haría la siguiente operación: $200 + (30min x $20).

2. Aplicamos distintos cargos fijos por pérdida de vehículo, los cuales son:
    - $50.000 para bicicleta común
    - $100.000 para bicicleta eléctrica.
    - $150.000 para monopatín eléctrico.

3. Una falla reportada no cambia el estado del vehículo: este sigue disponible hasta que un operador genere una orden de mantenimiento, salvo que alcance 3 reportes dentro de 30 días, en cuyo caso pasa automáticamente a "en reparación". Si la falla se reporta durante un viaje en curso, el viaje continúa hasta que el usuario devuelve el vehículo en una estación, y el reporte se registra como cualquier otro.

4. El costo del plan mensual no está especificado por lo que fijamos un precio fijo de $15.000 mensuales. El plan no vence ni se renueva.

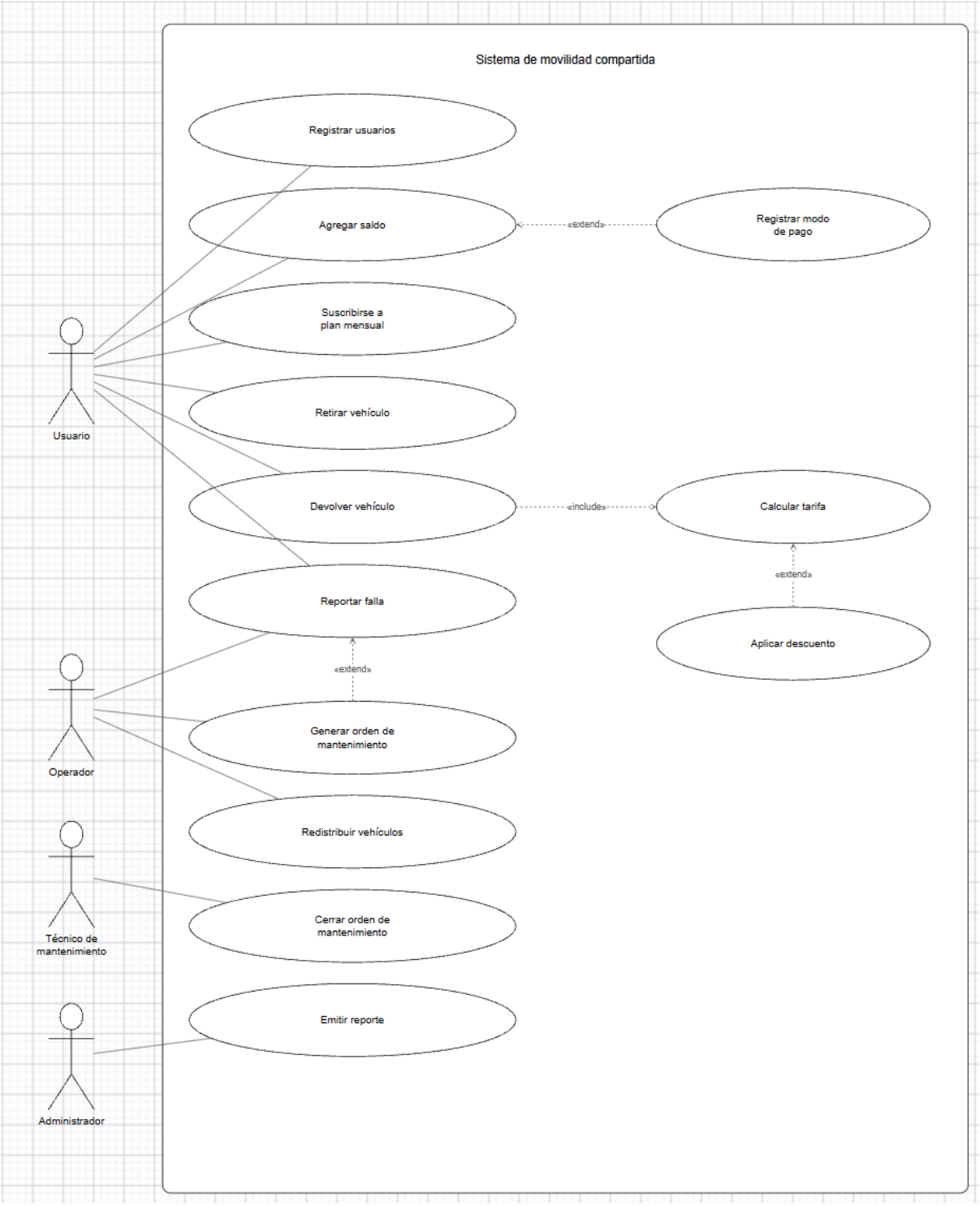
5. Si el usuario se excede de los 15 minutos del primer viaje o de los 30 minutos del plan mensual, deberá abonar los minutos excedentes al valor estándar del vehículo, sin cobrarse el cargo de desbloqueo.

6. Si un vehículo no se devuelve en 24 horas, el viaje se cierra con el cargo fijo por pérdida, que reemplaza a la tarifa por tiempo, y el vehículo pasa a "fuera de servicio".

7. Si la batería de un vehículo eléctrico baja del 15 % durante un viaje, el viaje continúa con normalidad; al devolverlo, el vehículo pasa a "batería baja" y deja de ofrecerse. Un vehículo con exactamente 15 % puede retirarse.

8. Un retiro cancelado dentro de los 2 minutos no cuenta como viaje: no consume el primer viaje gratuito ni uno de los 20 viajes del plan mensual.

Diagrama de casos de uso



[Diagrama de casos de uso (draw.io)]

Aclaración: una vez abierto el diagrama en Google Drive, tocar la opción de “Abrir con Draw.io” para una mejor visualización.

Actores del sistema

- **Usuario:** Persona registrada que utiliza los vehículos y gestiona su saldo.

- **Operador del sistema:** Encargado de redistribuir vehículos, registrar fallas y generar órdenes de mantenimiento
- **Técnico de mantenimiento:** Encargado de realizar las reparaciones de los vehículos.
- **Administrador:** Encargado de emitir reportes del sistema.

Casos de Uso y Descripciones

- **Registrar usuario**
    - **Actor:** Usuario.
    - **Precondición:** El usuario debe dar datos personales válidos.
    - **Flujo principal:** El usuario se registra en el sistema con sus datos básicos.
    - **Flujos alternativos:**
        - 1. Si alguno de los datos introducidos es erróneo el sistema avisará al usuario.

- **Retirar vehículo**
    - **Actor:** Usuario.
    - **Precondición:** El usuario debe estar habilitado (su saldo no debe ser negativo), tener el saldo suficiente para realizar un viaje de 30 minutos, no tener otro viaje abierto y el vehículo debe estar "disponible". En caso de que el vehículo sea eléctrico, este debe tener un porcentaje de batería igual o superior al 15%.
    - **Flujo principal:** El usuario retira un vehículo, el sistema registra el inicio, el recorrido y la estación origen.
    - **Flujos alternativos:**
        - 1. Si el saldo es insuficiente o tiene un viaje abierto el sistema le avisará al usuario que no podrá retirar un vehículo.
        - 2. Si no hay vehículos disponibles en la estación el sistema avisará al usuario.
        - Si el usuario cancela el retiro dentro de los primeros 2 minutos, el sistema anula el viaje y no se cobra.

- **Devolver vehículo**
    - **Actor:** Usuario.
    - **Relación:** Tiene una relación de include con el caso de uso “Calcular tarifa”.
    - **Precondición:** El usuario tiene un viaje abierto con un vehículo en estado “en uso” y se encuentra en una estación registrada.
    - **Flujo principal:** Al terminar el viaje, el usuario devuelve el vehículo en una estación con capacidad disponible. En ese momento se utiliza el caso de uso “Calcular tarifa”.
    - **Flujos alternativos:**
        - 1. Si el usuario intenta devolver el vehículo y la estación destino está llena, el sistema le da 10 minutos gratuitos para dirigirse a otra estación para devolverlo.

- **Calcular tarifa**
    - **Actor:** Sistema
    - **Precondición:** El viaje debe haber finalizado o superado el tiempo límite.
    - **Flujo principal:** El sistema calcula el costo como (cargo de desbloqueo + (precio por minuto × minutos de uso)), cambiando el precio por minuto dependiendo del tipo de vehículo. Luego,se verifican las promociones y se aplica la que corresponda, cobrándole el monto final al saldo del usuario
    - **Flujos alternativos:**
        - Si el viaje supera las 2 horas permitidas, se cobra un recargo del 50% por cada minuto excedente.
        - Si el vehículo no es devuelto pasadas 24 horas, el sistema lo marca como “Perdido” y cobra un cargo penalizador.

- **Reportar falla**
    - **Actores:** Usuario, Operador del sistema.
    - **Precondición:** El vehículo tiene que estar registrado e identificado en la plataforma.
    - **Flujo principal:** El usuario o el operador identifica una falla en un vehículo a través de la aplicación o del sistema. Ingresa el código del vehículo, selecciona el tipo de falla y añade una observación. El sistema registra el reporte. El estado del vehículo no cambia, salvo que se acumulen 3 reportes dentro de 30 días: en ese caso pasa a "en reparación".

    - **Flujos alternativos:**
        - 1. Si la falla se reporta durante un viaje en curso, el viaje continúa hasta que el usuario deje el vehículo en la estación más cercana.

- **Generar orden de mantenimiento**
    - **Actor:** Operador del sistema.
    - **Precondición:** Que exista un reporte de falla asociado a un vehículo.
    - **Flujo principal:** El operador revisa la lista de reportes pendientes y genera una orden de mantenimiento asignando la prioridad y el técnico responsable. El vehículo pasa al estado "en reparación".
    - **Flujos alternativos:**
        - 1. Si el operador verifica que no hay falla real, cancela el reporte y devuelve el vehículo al estado "disponible".
        - 2. Si el vehículo llega a 3 reportes de falla dentro de 30 días, la orden se genera automáticamente

- **Cerrar orden de mantenimiento**
    - **Actor:** Técnico de mantenimiento.
    - **Precondición:** Existe una orden de mantenimiento asignada.
    - **Flujo principal:** El técnico selecciona la orden de mantenimiento, registra su legajo, la fecha, las tareas realizadas y repuestos utilizados. Tras confirmar el arreglo, el sistema cierra la orden y el vehículo vuelve a estar "disponible".
    - **Flujos alternativos:**

        - 1. Si faltan repuestos para completar la reparación, el técnico deja la orden en espera y el vehículo sigue en mantenimiento.
        - 2. Si el vehículo no se puede reparar, se solicita la baja y pasa al estado "fuera de servicio".

- **Redistribuir vehículos**
    - **Actor:** Operador del sistema.
    - **Precondición:** Se da cuando una estación activa una alerta por tener menos del 20% o más del 90% de su capacidad ocupada.
    - **Flujo principal:** El operador atiende una alerta de ocupación (menos del 20% o más del 90%). Selecciona la estación origen, la estación destino y los vehículos a trasladar. Al confirmar, el sistema actualiza la ubicación y disponibilidad de las unidades.
    - **Flujos alternativos:**
        - 1. Si la estación destino se llena durante el trayecto, el operador redirige los vehículos a otra cercana.

- **Emitir reportes**
    - **Actor:** Administrador.
    - **Precondición:** El sistema debe contar con un registro histórico de datos.
    - **Flujo principal:** El administrador selecciona el tipo de informe deseado (uso por estación, viajes por usuario o vehículos con más fallas) y aplica los filtros correspondientes. El sistema procesa los datos y muestra el reporte solicitado.
    - **Flujos alternativos:**
        - 1. Si los filtros no devuelven datos, el sistema notifica que no hay registros disponibles para ese período.
        - 2. El administrador puede optar por exportar el reporte generado a un archivo externo.
- **Agregar saldo**
    - **Actor**: Usuario.
    - **Relación:** Registrar medio de pago extiende a Agregar saldo cuando el usuario no tiene un medio de pago asociado.
    - **Precondición:**El usuario debe contar con una cuenta.
    - **Flujo principal**: El usuario selecciona el monto a cargar deseado y se deposita en su cuenta.
    - **Flujos alternativos**:
        - 1. Si el importe es menor o igual a 0, el sistema rechaza la operación y avisa al usuario.
        - 2. Si el medio de pago es rechazado, no se acredita el monto y se avisa al usuario.

- **Registrar medio de pago**
    - **Actor:** Usuario.
    - **Precondición:** el usuario debe contar con una cuenta.
    - **Flujo principal:** El usuario selecciona el medio de pago deseado y se guarda para futuras operaciones.

    - **Flujos alternativos:**
        - 1. Si el usuario no existe el sistema avisará al usuario que no se puede registrar el medio de pago.

- **Suscribirse a plan mensual**
- **Actor:** Usuario.
- **Precondición:** El usuario está registrado, no tiene un plan mensual vigente y su saldo (o su medio de pago asociado) cubre el costo del plan ($15.000).
- **Flujo principal:** El usuario elige suscribirse al plan mensual. El sistema debita $15.000 de su saldo, registra el movimiento y activa el plan con 20 viajes de hasta 30 minutos cada uno.
- **Flujos alternativos:**
    - Si el saldo no alcanza y no tiene un medio de pago asociado, el sistema rechaza la suscripción e informa que debe cargar saldo.

Aplicar descuento

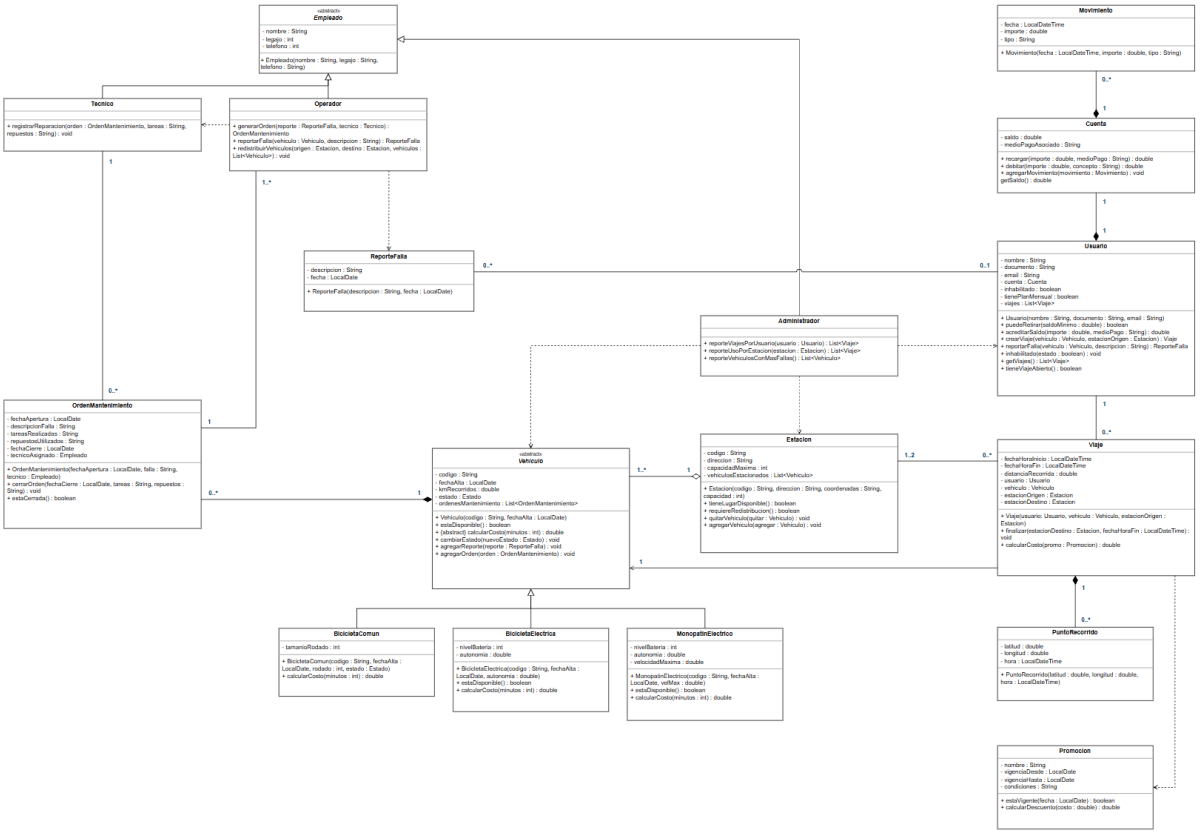
- **Actor:** Sistema.
- **Relación:** Extiende el caso de uso “Calcular tarifa” cuando el usuario tiene una promoción aplicable.
- **Precondición:** El viaje finalizó y el usuario tiene al menos una promoción aplicable (primer viaje o plan mensual con viajes disponibles).
- **Flujo principal:** Durante el cálculo de la tarifa, el sistema evalúa las promociones del usuario, calcula el descuento de cada una y aplica solo la más favorable, porque no se acumulan. El monto resultante se debita del saldo y, si se usó el plan mensual, se descuenta un viaje del cupo.
- **Flujos alternativos:**
    1. Si el viaje supera los 15 minutos del primer viaje o los 30 minutos del plan, se cobran los minutos excedentes al precio estándar del vehículo, sin cargo de desbloqueo.
    2. Si el plan mensual ya usó sus 20 viajes, no se considera y se evalúan las demás promociones. Si no queda ninguna, se cobra la tarifa completa.

Diagramas de clases

Los siguientes diagramas de clases representan el modelado del sistema propuesto. El diagrama inicial muestra cómo se plantea el sistema sin aplicar patrones GRASP ni principios SOLID. En él podemos ver clases que se identificaron en una primera revisión,

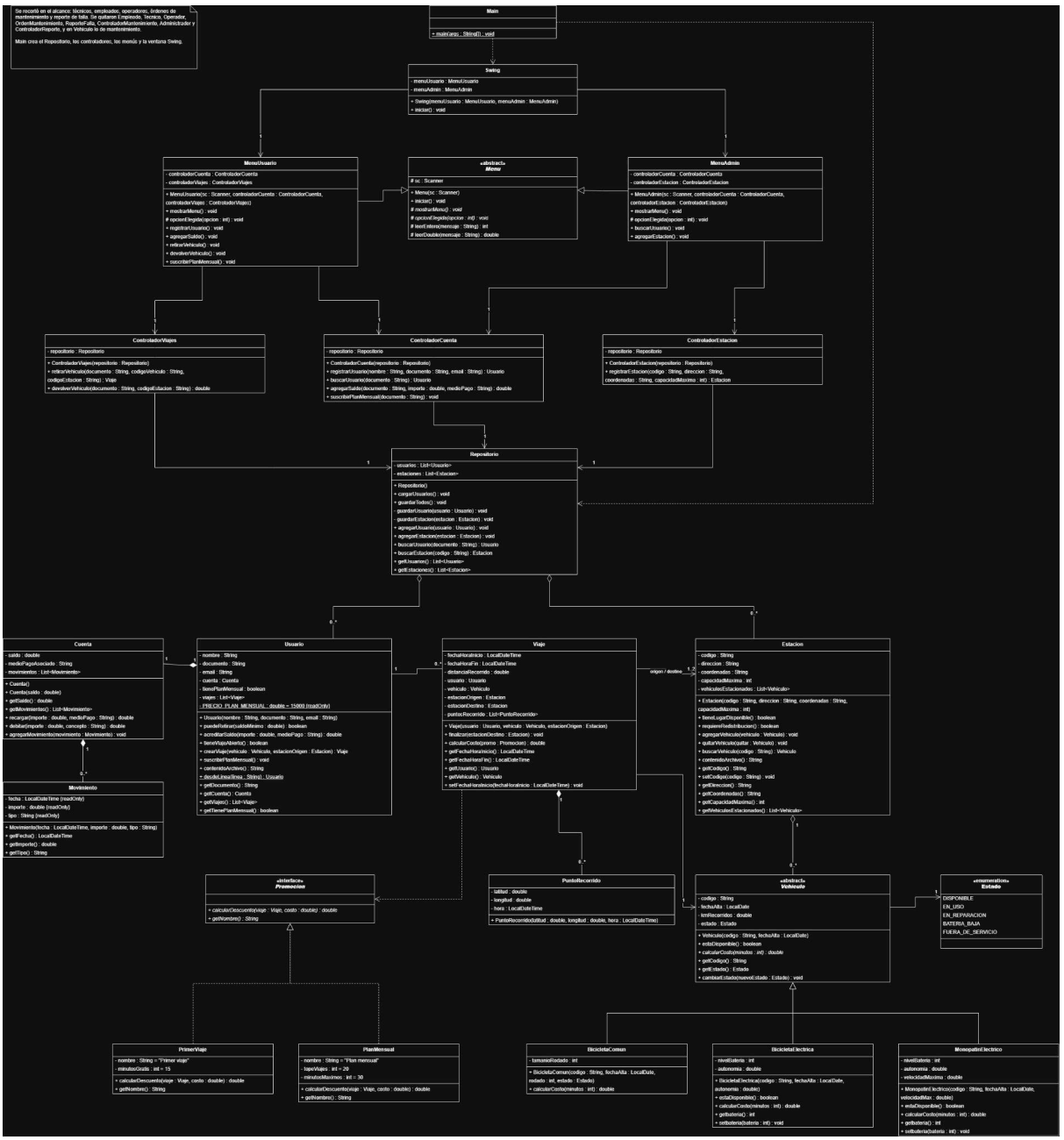
tales como Empleado y Vehículo que resultan ser abstractas, Cuenta, Usuario, ReporteFalla, Movimiento, Estacion, Viaje y PuntoRecorrido.
El diagrama final muestra un sistema más robusto y flexible, que incorpora distintos patrones y principios tales como Experto en información, Creador, Controlador, Polimorfismo y también en menor medida Fabricación pura. Entre los cambios respecto del inicial se encuentran la incorporación de clases como los controladores, MenuConsola y Repositorio, y la eliminación del atributo inhabilitado, ya que se deduce del saldo y de los viajes del usuario.

Diagrama de clases inicial:



[Diagrama de clases inicial (draw.io)]

Diagrama de clases final:



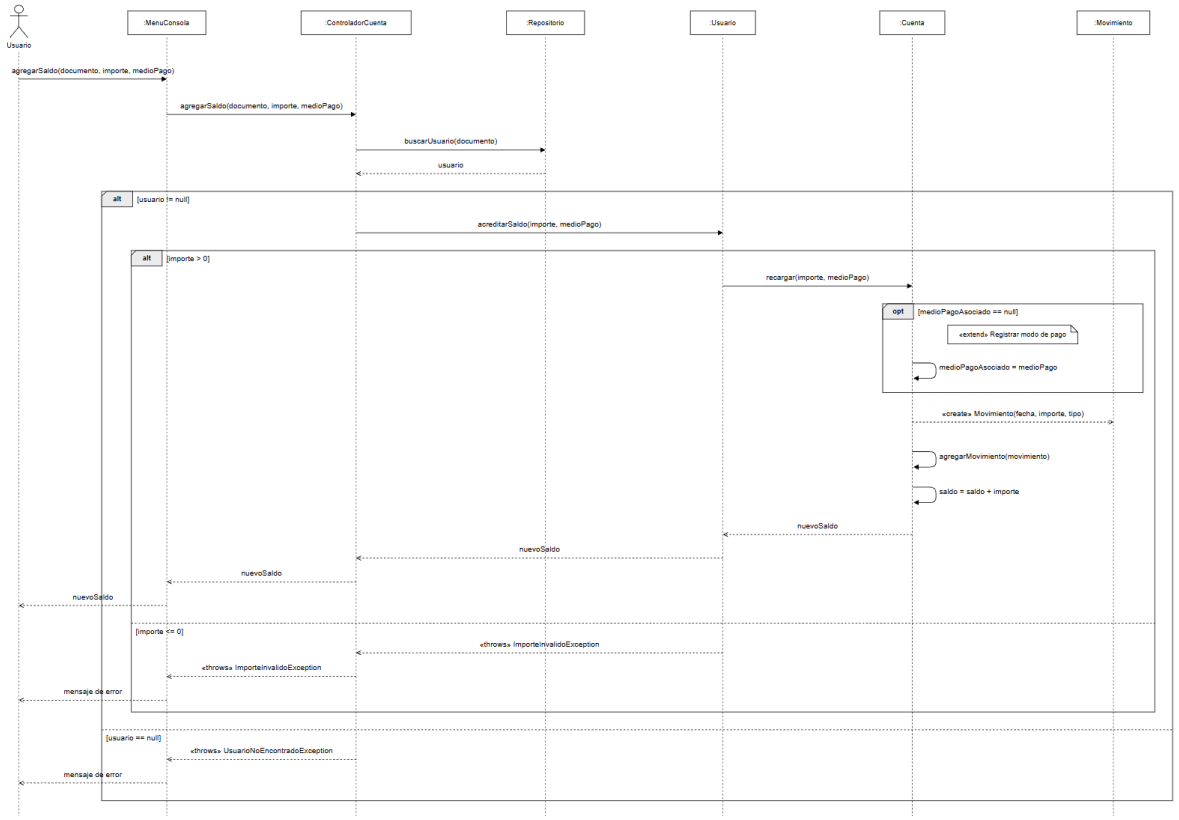
[Diagrama de clases final (draw.io)]

Aclaración: una vez abiertos los diagramas en Google Drive, tocar la opción de “Abrir con Draw.io” para una mejor visualización.

Diagramas de secuencia

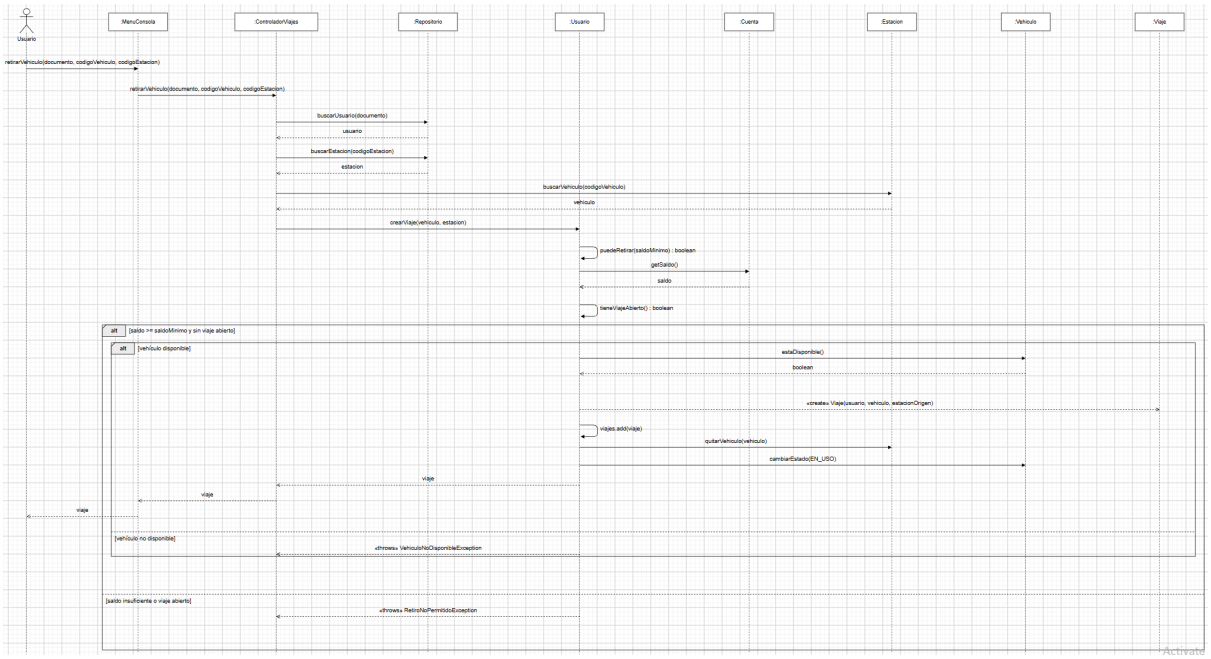
Los dos casos de uso seleccionados son “Retirar vehículo” y “Agregar saldo”. Estos mostraran el flujo de estas dos operaciones, con sus respectivos flujos alternativos y excepciones teniendo como base el diagrama de clases final.

Diagrama de secuencia del caso de uso agregar saldo:



[secuencia agregar_saldo (draw.io)]

Diagrama de secuencia del caso de uso retirar vehículo:



[secuencia retirar_vehiculo (draw.io)]

Aclaración: una vez abiertos los diagramas en Google Drive, tocar la opción de “Abrir con Draw.io” para una mejor visualización.

Patrones GRASP y principios SOLID

Durante la revisión del modelo conceptual inicial, se detectaron desafíos de diseño que podrían comprometer la escalabilidad del sistema en etapas posteriores. En particular, la concentración de responsabilidades en clases controladoras y la dependencia directa de la lógica de negocio respecto a tipos concretos de vehículos y esquemas tarifarios representan un riesgo de alto acoplamiento y baja cohesión.

A fin de eliminar estos problemas y conseguir un código robusto, flexible y abierto a la extensión (sin necesidad de modificar clases existentes), fundamentamos la aplicación de los principios SOLID y patrones GRASP, justificando cómo cada uno aporta a la calidad estructural del software.

Aclaración: las siguientes justificaciones están basadas en clases que aparecen en el diagrama de clases final.

GRASP

Para los patrones GRASP decidimos elegir Creador, Polimorfismo, Controlador, Experto y, en menor medida, Fabricación pura. A continuación pasaremos a explicar el porqué de esta elección.

Para el patrón Creador nos preguntamos qué clase debía encargarse de crear los objetos Viaje y Movimiento. Si los creara el controlador o MenuConsola, estas clases tendrían que conocer todos los datos necesarios para construirlos y dependería de más clases de las necesarias. Según este patrón, una clase B debe crear instancias de una clase A cuando B contiene o agrega a A, lo registra, lo usa de cerca o tiene los datos para inicializarlo.

Aplicamos este criterio en dos clases. Usuario crea el Viaje mediante crearViaje(), ya que guarda la lista de sus viajes y tiene los datos para construirlo: él mismo, el vehículo y la estación de origen. Esto se ve en el diagrama de secuencia de Retirar vehículo, donde ControladorViajes le pide al Usuario que cree el viaje en lugar de hacerlo por su cuenta. Por otro lado, Cuenta crea los objetos Movimiento: dentro de recargar() genera un movimiento de crédito y dentro de debitar() uno de débito, y en ambos casos lo registra con agregarMovimiento(). Además, la relación entre Cuenta y Movimiento es una composición, por lo que un movimiento no puede existir fuera de su cuenta.

La ventaja de aplicar Creador es que la creación de cada objeto queda en la clase que ya está relacionada con él, lo que mantiene bajo el acoplamiento. Ni los controladores ni la interfaz necesitan saber cómo se construyen Viaje o Movimiento, y si cambia alguno de sus constructores solo hay que modificar Usuario o Cuenta.

Para el patrón de Polimorfismo decidimos hacer de la clase Vehiculo una abstracta con un método abstracto (calcularCosto()), la cual será heredada por BicicletaComun, BicicletaElectrica y MonopatinElectrico, cada una con una implementación propia para el

método antes mencionado. Sin este patrón, el cálculo del costo tendría que preguntar qué tipo de vehículo es y habría que tocarlo con cada tipo nuevo. Con polimorfismo, Viaje invoca calcularCosto() sin saber qué tipo de vehículo es. Otra sección en donde se puede apreciar con más fuerza este patrón es con Promocion: pudimos observar que, al ser una clase común, implementar las promociones que necesitaba el programa (PrimerViaje y PlanMensual) sería muy rígido, por lo que Promocion se convirtió en una interfaz y las clases PrimerViaje y PlanMensual implementan sus métodos.

El patrón Controlador decidimos aplicarlo en cuatro clases: ControladorCuenta, ControladorViajes, ControladorMantenimiento y ControladorReporte, cada una encargada de un área del sistema. Este patrón indica que el primer objeto que recibe un pedido de la interfaz, sin ser parte de ella, debe ser un controlador que coordine la operación y delegue el trabajo en las clases del dominio. En nuestro diseño, MenuConsola solo recibe los datos que ingresa el usuario y los envía al controlador correspondiente; este utiliza el Repositorio para convertir esos datos (por ejemplo, el documento del usuario o el código de la estación) en objetos y luego delega en ellos la lógica del negocio. Esto se puede ver en el diagrama de secuencia de Retirar vehículo, donde ControladorViajes busca el Usuario y la Estacion y le pide al Usuario que cree el viaje mediante crearViaje(), sin validar el saldo ni crear el Viaje por su cuenta. Sin este patrón, MenuConsola tendría que conocer y manejar directamente las clases del dominio, mezclando la interfaz con la lógica del negocio y haciendo que un cambio en una afecte a la otra. Además, al dividir los controladores por área en lugar de usar uno solo que se ocupe de todo, evitamos una clase con demasiadas responsabilidades, lo que mejora la cohesión y reduce el acoplamiento, uno de los riesgos detectados en el modelo inicial.

Para el patrón Experto decidimos asignar cada responsabilidad a la clase que posee la información necesaria para cumplirla. Un primer ejemplo es Usuario: al tener la lista de sus viajes es quien determina si tiene un viaje abierto mediante tieneViajeAbierto(), y al conocer su Cuenta puede consultar el saldo con getSaldo() para decidir en puedeRetirar() si cumple con el mínimo para retirar un vehículo. Otro ejemplo es Estacion, que al conocer los vehículos estacionados es la encargada de buscarlos (buscarVehiculo()), quitarlos (quitarVehiculo()) y agregarlos (agregarVehiculo()), y también Cuenta, que al contener su saldo y sus movimientos es la responsable de recargar() y debitar(). Sin este patrón, esa información tendría que obtenerse desde afuera; por ejemplo, el controlador debería recorrer los viajes del usuario para saber si hay uno abierto, lo que lo haría depender de la estructura interna de las clases y aumentaría el acoplamiento. Al aplicarlo, cada clase se encarga de lo que conoce, lo que mantiene una alta cohesión y hace que un cambio en los datos de una clase solo afecte a esa clase.

Por último, aplicamos en menor medida el patrón Fabricación pura con la clase Repositorio. Este patrón propone crear una clase que no representa un concepto del dominio cuando asignar una responsabilidad a una clase del dominio bajaría su cohesión o aumentaría el acoplamiento. En nuestro sistema, alguien debía encargarse de almacenar los usuarios y las estaciones y de buscarlos a partir de los datos que ingresaba el usuario, como su documento o el código de una estación. Si esta tarea quedará en Usuario o en Estacion, estas clases mezclarían su lógica de negocio con la gestión de datos. Si quedara en los controladores, cada uno tendría su propia lista y se repetiría código. Por eso creamos Repositorio, una clase inventada que no existe en el enunciado y cuya única función es

guardar y buscar objetos. Los controladores lo usan para obtener el Usuario y la Estacion antes de delegar la operación, como se ve en el diagrama de secuencia de Retirar vehículo. La ventaja es que las clases del dominio mantienen una alta cohesión. Además, si más adelante los datos se guardan en un archivo o una base de datos, solo habría que modificar Repositorio.

SOLID

Nosotros decidimos utilizar los principios S, L y O de SOLID. Estos principios están enfocados en las relaciones entre las clases, sus responsabilidades y cómo las clases funcionan entre ellas para producir un código más flexible, extensible y mantenible a lo largo del tiempo.

Primero implementamos responsabilidad única (S), la cual describe cómo cada clase debe tener un propósito y estar relacionada con una única actividad o área para no crear una clase que se ocupe de todo el funcionamiento del código, mejorando la cohesión y la división de trabajos entre las clases. En nuestra propuesta se puede ver en clases como Movimiento, que se encarga únicamente de representar una operación de saldo; Cuenta, que manipula únicamente el saldo y sus movimientos; y Repositorio, que tiene como función almacenar y buscar datos. Por ejemplo, si Cuenta también fuera responsable de calcular tarifas, cualquier cambio en las reglas de precios obligaría a modificar una clase cuya responsabilidad principal es administrar el saldo y sus movimientos.

También aplicamos el principio de sustitución de Liskov (L). Este principio indica que los objetos de una subclase deben poder reemplazar a los de su clase padre sin que el programa deje de funcionar correctamente. En nuestro diseño, Viaje trabaja con un Vehiculo sin saber si es una BicicletaComun, una BicicletaElectrica o un MonopatinElectrico. Cualquiera de las tres responde a calcularCosto() respetando lo que se espera de ese método, que es devolver el costo según los minutos de uso. Lo mismo ocurre con Promocion: Viaje puede trabajar con un PrimerViaje o un PlanMensual, y en ambos casos calcularDescuento() devuelve un descuento válido sin que Viaje tenga que preguntar de qué tipo es. Si alguna subclase no respeta este principio, por ejemplo si devolviera un costo negativo o lanzara un error, habría que agregar condiciones según el tipo y se perdería el polimorfismo. Al respetarlo, cualquier subclase puede usarse en lugar de su clase padre y el sistema sigue funcionando igual.

Finalmente, hicimos uso del principio de abierto/cerrado (O) para que las clases estén abiertas a la extensión pero cerradas a la modificación. Esto lo podemos ver, por ejemplo, en el uso de la interfaz Promocion, la cual se relaciona con Viaje para calcular el precio final de este. Con esta interfaz nosotros podemos agregar nuevas promociones sin tener que cambiar la clase Viaje. Este principio está relacionado con el principio GRASP de variaciones protegidas enfocado en las abstracciones donde sabemos que el código es probable que cambie a lo largo del tiempo.